

Documenti autorizzati: due fogli manoscritti A4 fronte retro.

1 Esercizio 1

```
//Variabili condivise
int a=0;
int b=0;
```

```
--Process P
while(true){
p1:   Sezione Non Critica
p2:   b=2;
p3:   while(a!=0){
p4:     a=a-3;}
p5:   Sezione Critica
p6:   b=0;
}
```

```
--Process Q
while(true){
q1:   Sezione Non Critica
q2:   a=2;
q3:   while(b!=0){
q4:     b=b-3;}
q5:   Sezione Critica
q6:   a=0;
}
```

Figura 1: Algoritmo di mutua esclusione

Consideriamo l'algoritmo di mutua esclusione proposto alla figura 1.

1. Disegnare l'inizio dello diagramma degli stati per questo algoritmo (massimo 10 stati).
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo la proprietà di assenza di deadlock? Giustificare.
4. Cosa si può dire sulla proprietà di mutua esclusione se prendiamo per il processo P il codice proposto alla figura 2? Giustificare.

```
--Process P
while(true){
p1:   Sezione Non Critica
p2:   b=2;
p3:   while(a!=0){
p4:     a=a-2;}
p5:   Sezione Critica
p6:   b=0;
}
```

Figura 2: Cambiamento nel process P

2 Esercizio 2

In un palazzo, c'è un ascensore che va dal piano terra all'ultimo piano senza fermarsi avanti e dietro. La capacità massima di questo ascensore è di 5 persone e non parte se è vuoto.

Il comportamento degli utenti è lo seguente. Al inizio, si trovano al piano terra, poi eseguono in ciclo:

1. Aspettare che l'ascensore arrivi al piano in cui si trovano e di potere.
2. Salgono in ascensore
3. Aspettano che l'ascensore arrivi al piano (terra o ultimo, a seconda della direzione dell'ascensore).

4. Scendono dall'ascensore.

Lo comportamento dell'ascensore è il seguente. Al inizio è al piano terra, poi esegue in ciclo:

1. Aspetta che almeno un utente salga.
2. Va al piano di destinazione (piano terra o ultimo piano a seconda della direzione).
3. Aspetta di essere vuoto.

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, tenendo in considerazione che vogliamo un processo leggero per l'ascensore e per ogni utente.

3 Esercizio 3

```
--Process Pi per i in {0,1,2}
    boolean CS=false;
    int ack1=0;
    int ack2=0;

    while(true){
p1:   Sezione Non Critica
p2:   CS=true;
p3:   while(ack1+ack2!=2){
p4:       send(req,i)->P(i+1)%3;
p5:       send(req,i)->P(i+2)%3;}
p6:   Sezione Critica
p7:   ack1=0;
p8:   ack2=0;
p9:   CS=false;
    }

--Gestione dei messaggi di Pi
--per i in {0,1,2}
case(req,j):
    [if(!CS){
        send(go,i)->P(j);}
    ]
    case(go,(i+1)%3): [ack1=1;]
case(go,(i+2)%3):[ack2=1;]
```

Figura 3: Algoritmo di mutua esclusione con message passing

Consideriamo l'algoritmo con message passing proposto sulla figura 3 per 3 processi. Assumiamo che il codice fra [...] si esegue in modo atomico sul sito di ciascun processo.

1. Descrivere in qualche righe come funziona questo algoritmo.
2. Verifica questo algoritmo la proprietà di mutua esclusione? Giustificare.
3. Verifica questo algoritmo l'assenza di deadlock? Giustificare.
4. Verifica questo algoritmo la proprietà di assenza di starvation? Giustificare.

4 Esercizio 4

Per un esame di programmazione, sono disponibili due laboratori Lab1 e Lab2. Lab1 dispone di 10 posti mentre Lab2 ne dispone di 7. Non possono esserci più studenti dei posti disponibili nei laboratori e al massimo possono esserci 14 studenti nei due laboratori. Uno studente prova prima a sedersi nel Lab1 e se non ci sono posti disponibili, va in Lab2 e se non ci sono posti o ci sono più di 14 studenti seduti aspetta che si libera un posto. Poi fa l'esame e quando ha finito esce dal laboratorio.

Proponete in JAVA, C o pseudo-codice una modellazione di questo sistema, tenendo in considerazione che vogliamo un processo leggero per ciascun studente.